Pandas Assignment — Solutions

Solutions for all listed tasks. The assignment's initial DataFrame does not include City, although later questions require it; the solutions add an explicitly labelled example City column for those tasks.

1. Setup and version

Python / SQL code

```
import pandas as pd
print(pd.__version__)
```

Sample output

```
Prints the installed Pandas version, e.g. 2.x.x.
```

2. Create DataFrame and inspect

Python / SQL code

```
data = {
    "Name": ["Alice", "Bob", "Charlie", "David"],
    "Age": [24, 27, 22, 32],
    "Department": ["HR", "IT", "Finance", "Marketing"],
    "Salary": [50000, 60000, 55000, 65000]
}
df = pd.DataFrame(data)
print(df.head(3))
print(df.columns)
print(df.dtypes)
print(df.describe(include="all"))
```

Sample output

```
Name  Age Department  Salary
0   Alice   24         HR   50000
1     Bob   27         IT   60000
2 Charlie   22    Finance   55000

Columns: Name, Age, Department, Salary
Age and Salary are int64; Name and Department are object.
describe() prints count, mean, std, min, quartiles and max for numeric columns.
```

3. Load CSV file

The assignment's sample CSV can be downloaded separately. Save it as hw_200.csv before running this code.

Python / SQL code

```
url = "https://people.sc.fsu.edu/~jburkardt/data/csv/hw_200.csv"
csv_df = pd.read_csv(url, header=None)
print(csv_df.head())
print(csv_df.describe(include="all"))
```

Sample output

```
Displays first 5 rows and descriptive statistics of the downloaded CSV.
Exact output depends on availability and contents of the URL.
```

4. Select Name and Salary; filter Age > 25

Python / SQL code

```
print(df[["Name", "Salary"]])
print(df[df["Age"] > 25])
```

Sample output

```
Name  Salary
0    Alice   50000
1      Bob   60000
2  Charlie   55000
3    David   65000

    Name  Age Department  Salary
1    Bob   27         IT   60000
3  David   32  Marketing   65000
```

5. Filter City New York or Chicago

The original four-row DataFrame has no City column. This task needs a City field; here is a clearly marked example extension.

Python / SQL code

```
df_city = df.assign(City=["New York", "Chicago", "Boston", "New York"])
print(df_city[df_city["City"].isin(["New York", "Chicago"])])
```

Sample output

```
Rows for Alice (New York), Bob (Chicago), and David (New York) are displayed.
```

6. Add Bonus and Total Compensation; replace City

Python / SQL code

```
df_city["Bonus"] = df_city["Salary"] * 0.10
df_city["Total Compensation"] = df_city["Salary"] + df_city["Bonus"]
df_city["City"] = df_city["City"].replace("New York", "NYC")
print(df_city[["Name", "City", "Salary", "Bonus", "Total Compensation"]])
```

Sample output

```
Alice: NYC, Salary 50000, Bonus 5000, Total 55000
Bob: Chicago, Salary 60000, Bonus 6000, Total 66000
Charlie: Boston, Salary 55000, Bonus 5500, Total 60500
David: NYC, Salary 65000, Bonus 6500, Total 71500
```

7. Sort by Age and reset index

Python / SQL code

```
sorted_df = df_city.sort_values("Age", ascending=True).reset_index(drop=True)
print(sorted_df[["Name", "Age"]])
```

Sample output

```
Name  Age
0 Charlie   22
1   Alice   24
2     Bob   27
3   David   32
```

8. Salary aggregations

Python / SQL code

```
print("Mean:", df_city["Salary"].mean())
print("Median:", df_city["Salary"].median())
print("Std:", df_city["Salary"].std())
```

Sample output

```
Mean: 57500.0
Median: 57500.0
Std: 6454.972243679028
(Pandas std uses sample standard deviation, ddof=1.)
```

9. Group by City; highest average total compensation

Python / SQL code

```
city_summary = df_city.groupby("City")[["Salary", "Bonus", "Total Compensation"]].mean()
print(city_summary)
print("Highest average total compensation:",
      city_summary["Total Compensation"].idxmax())
```

Sample output

```
Boston: average total compensation 60500
Chicago: 66000
NYC: 63300
Highest average total compensation: Chicago
```

10. Missing values

Python / SQL code

```
data = {
    "Name": ["Alice", "Bob", "Charlie", None, "Eva"],
    "Age": [24, 27, None, 32, 29],
    "City": ["New York", None, "Chicago", "Houston", "Phoenix"],
    "Salary": [70000, 80000, None, 90000, 75000]
}
df_missing = pd.DataFrame(data)
print(df_missing.isna().sum())
df_missing["City"] = df_missing["City"].fillna("Unknown")
df_missing = df_missing.dropna(subset=["Salary"]).copy()
```

```
df_missing["Age"] = df_missing["Age"].fillna(df_missing["Age"].mean())
print(df_missing)
```

Sample output

```
Missing counts before handling:
Name      1
Age       1
City      1
Salary    1

After dropping missing Salary, Charlie's row is removed.
City missing value becomes Unknown.
Remaining missing Age is filled with mean of remaining known ages:
(24 + 27 + 32 + 29) / 4 = 28.0.
Rows retained: Alice, Bob, David (missing Name), Eva.
```